

TD n°2 : Réseaux de neurones multicouches

Ce TD illustre le fonctionnement d'un perceptrons multi-couches sur un problème de classification et explore l'impact des fonctions d'activation des neurones de la couche cachée, du nombre de neurones cachés et du nombre de couches cachées.

Préparation des données

Les données employées sont constituées d'un ensemble déjà connu de 150 observations de fleurs d'iris. Pour chaque observation sont disponibles la longueur et la largeur des sépales (en cm), la longueur et la largeur des pétales (en cm), ainsi que la variété d'iris (Iris Setosa, Iris Versicolo(u)r, Iris Virginica). L'objectif est de prédire à quelle classe appartient un échantillon d'iris (classe correspondant à la variable dite *expliquée*, ici la variété) à partir des quatre variables dites *explicatives* (longueurs et largeurs des sépales et des pétales).

Dans Scikit-learn, on trouve les données Iris déjà présentes, chargeable directement avec

```
# importation bibliotheque numpy
import numpy as np
# importation fonction chargement donnees iris
from sklearn.datasets import load_iris
donnees = load_iris()
# affichage 3 premieres observations
donnees.data[0:3,:]
# dimensions matrice observations
donnees.data.shape
# affichage etiquettes classe 3 premieres observations
donnees.target[0:3]
# dimensions vecteur etiquettes de classe
donnees.target.shape
```

Ou alors, pour illustrer un cas plus général, télécharger directement à partir du site le fichier de données (en format CSV), soit à partir de la page

```
http://archive.ics.uci.edu/dataset/53/iris
```

soit, par le biais de la commande `wget`

```
%%bash
wget https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data
```

et en lire les valeurs des quatre variables explicatives

```
# valeurs separees par virgule, colonnes 0 a 3, y compris 1ere ligne
data = np.loadtxt('iris.data', delimiter=',', usecols=[0,1,2,3], skiprows=0)
```

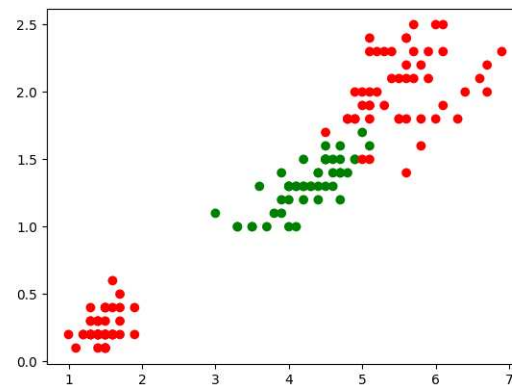
Les étiquettes étant sous forme textuelle dans le fichier `iris.data` (c'est à dire Iris-setosa, Iris-versicolor et respectivement Iris-virginica) nous tirons profit du fait que ces trois classes se succèdent dans le fichier de données et qu'il y a exactement 50 observations par classe pour générer directement des étiquettes de classe. Nous chercherons d'abord à construire des modèles permettant de séparer les iris en deux classes, Iris-setosa et les autres (Iris-versicolor et Iris-virginica). Nous utilisons alors une étiquette 1 pour Iris-setosa et une étiquette 0 pour les autres :

```
# generer etiquettes de classe
l1c = np.ones(50, dtype=int)
l2c = np.zeros(50, dtype=int)
target = np.concatenate((l2c, l1c, l2c))
```

Dans le cas général il faudra employer une solution de conversion des étiquettes de classe adaptée à leur format. Une bibliothèque python qui propose de nombreuses fonctionnalités de lecture et transformation des données est *panda*. Nous pouvons visualiser les observations en nous limitant à deux ou trois variables explicatives. Par exemple, les deux dernières (longueur et largeur des pétales, en cm) :

```
# suivant la classe, les points affiches sont rouges ou verts (green)
cmp = np.array(['r','g'])
# importer les fonctions d'affichage 2D
import matplotlib.pyplot as plt
plt.figure()
plt.scatter(data[:,2], data[:,3], c=cmp[target], s=50, edgecolors='none')
plt.show()
```

Le résultat (qui devrait correspondre à la figure suivante) montre que le problème de séparation visé n'est pas trivial si on se limite aux deux premières variables explicatives.



Exercice 1 Affichez en 3D les observations en vous limitant aux trois dernières variables explicatives.

Perceptrons multi-couches (PMC) dans Scikit-learn

Dans Scikit-learn, les PMC sont implémentés dans les classes `MLPClassifier` pour la classification et `MPRegressor` pour la régression. Nous examinons dans la suite l'utilisation des PMC pour traiter des problèmes de classification. Les paramètres de `MLPClassifier` permettent de définir l'architecture du PMC (nombre de couches et de neurones dans chaque couche), les fonctions d'activation pour la(les) couche(s) cachée(s), la méthode d'optimisation (L-BFGS, SGD ou Adam avec leurs hyper-paramètres), les méthodes de régularisation (que nous examinerons dans une séance de TP ultérieure), etc. Le nombre d'entrées est donné par le nombre de variables explicatives et dépend de leur représentation (une variable nominale sera représentée par un codage one-hot et donc aura autant d'entrées spécifiques qu'elle a de modalités). Le nombre de sorties est égal au nombre de classes, même si dans les données utilisées une seule variable représente la classe par une valeur numérique différente. La fonction d'erreur (loss) employée est l'entropie croisée (*cross-entropy*, *log loss*, *logistic loss*).

Ensembles d'apprentissage et de test Séparons d'abord l'ensemble des observations disponibles en données d'apprentissage et données de test. Pour pouvoir répéter les expérimentations, il est utile d'initialiser explicitement le générateur pseudo-aléatoire avec `np.random.seed(...)`. La fonction `train_test_split` sépare les données en deux ensembles distincts, train et test, suivant la proportion de données de test indiquée avec `test_size`; la séparation est réalisée par échantillonnage aléatoire uniforme.

```
np.random.seed(10) # initialiser le g\en\érateur pseudo-al\éatoire pour pouvoir
                  # repeter l'expérience
from sklearn.model_selection import train_test_split
# X : var. explicatives, y : var. expliquée, train : apprentissage, test : test
X_train1, X_test1, y_train1, y_test1 = train_test_split(data, target, test_size=0.4)
```

Construction d'un modèle pour illustration Afin de pouvoir visualiser la frontière nonlinéaire de séparation entre classes, nous construisons d'abord un modèle avec des valeurs par défaut pour tous les hyper-paramètres sauf pour le nombre maximum d'itérations et l'optimiseur (la valeur par défaut de 200 étant insuffisante pour la méthode d'optimisation SGD - descente de gradient stochastique) en utilisant les deux premières variables explicatives. Noter que la fonction d'activation par défaut pour les neurones de la couche cachée est ReLU. Vous pouvez examiner les valeurs par défaut des autres paramètres.

```
from sklearn.neural_network import MLPClassifier
# donner une valeur a random_state rend possible la repetabilite
clf = MLPClassifier(solver='sgd', learning_rate_init=0.01, max_iter=10000, random_state=0)
# apprentissage avec les deux premieres variables
clf.fit(X_train1[:,2:4], y_train1)
```

Les taux de bon classement en apprentissage (train) et en test sont :

```
train_score = clf.score(X_train1[:,2:4], y_train1)
print("Le score en train est {}".format(train_score))
test_score = clf.score(X_test1[:,2:4], y_test1)
print("Le score en test est {}".format(test_score))
```

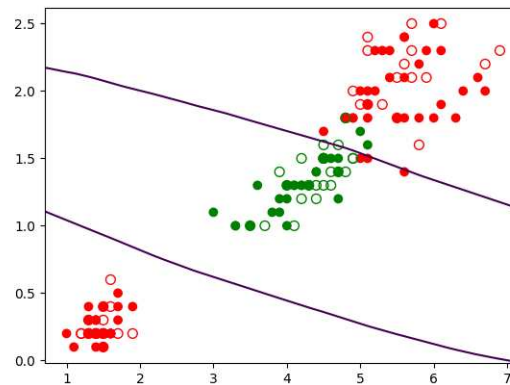
Nous affichons les résultats : observations d'apprentissage (points pleins), observations de test (points creux), frontière de séparation. Pour déterminer la frontière de séparation, nous définissons dans le domaine de variation des deux variables explicatives une grille dense, calculons les prédictions du modèle sur tous les points de la grille et affichons un contour qui correspond aux points du plan où l'indétermination est maximale.

```

# créer une nouvelle figure
plt.figure()
# afficher les nuages de points d'apprentissage (pleins) et de test (creux)
plt.scatter(X_train1[:,2], X_train1[:,3], c=cmap[y_train1], s=50, edgecolors='none')
plt.scatter(X_test1[:,2], X_test1[:,3], c='none', s=50, edgecolors=cmap[y_test1])
# calculer la probabilité de sortie du MLP pour tous les points du plan
nx, ny = 200, 200
x_min, x_max = plt.xlim()
y_min, y_max = plt.ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, nx), np.linspace(y_min, y_max, ny))
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])
Z1 = Z[:, 1].reshape(xx.shape)
# dessiner le contour correspondant à la fronti re proba = 0,5
plt.contour(xx, yy, Z1, [0.5])
plt.show()

```

Le r sultat (qui devrait correspondre   la figure suivante) montre la s paration trouv e.



Nous avons employ  la fonction `predict_proba` afin d'obtenir, pour chaque point de la grille d finie par `xx`, `yy`, la pr diction du r seau sous la forme de probabilit . Le r seau a deux neurones de sortie, un donne la probabilit  d'appartenance   la classe *Versicolor* et l'autre neurone la probabilit  d'appartenance   l'autre classe. Avec `Z.shape` il est possible de v rifier que pour chaque observation il y a bien deux probabilit s.

Exercice 2 Vérifiez que la somme des éléments de chaque ligne de Z est égale à 1 (la condition de normalisation est vérifiée).

Il est possible de visualiser les probabilités d'affectation à chaque classe comme des surfaces en 3 dimensions (le plan XY est celui des variables explicatives, l'axe Z correspond aux probabilités d'appartenance). Pour que la figure soit plus claire, nous ajoutons les observations avec scatter mais à mi-hauteur (plutôt que dans le plan $Z=0$).

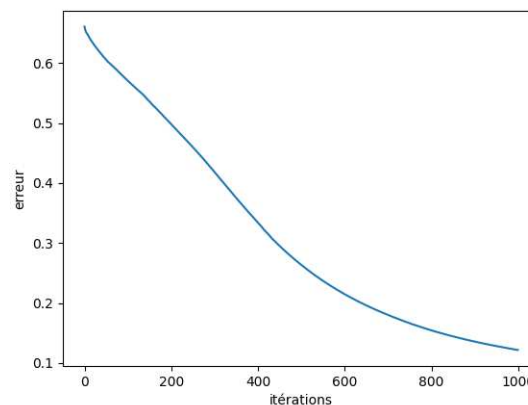
```
from matplotlib import cm
ax = plt.figure().add_subplot(projection='3d')
ax.scatter(X_train1[:,2],X_train1[:,3],0.5,c=cmp[y_train1],s=50)
ax.scatter(X_test1[:,2],X_test1[:,3],0.5,c=cmp[y_test1],s=50)
x_min, x_max = plt.xlim()
y_min, y_max = plt.ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, nx),np.linspace(y_min, y_max, ny))
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])
Z0 = Z[:, 0].reshape(xx.shape)
Z1 = Z[:, 1].reshape(xx.shape)
# probabilite d'appartenance a la classe Versicolor
surf1=ax.plot_surface(xx, yy, Z0, alpha=0.5, cmap=cm.Reds)
# probabilite d'appartenance a une autre classe
surf2=ax.plot_surface(xx, yy, Z1, alpha=0.5, cmap=cm.Greens)
fig.colorbar(surf1, shrink=0.5, aspect=5)
fig.colorbar(surf2, shrink=0.5, aspect=5)
plt.show()
```

Exécuter ce programme pour obtenir la figure associée.

Exercice 3 Quelle colonne de Z correspond à la probabilité d'appartenance à la classe *Versicolor* ?

Il est également possible de visualiser l'évolution de l'erreur d'apprentissage au fil des itérations car les valeurs sont conservées dans l'attribut `loss_curve_` du modèle obtenu :

```
plt.figure()
plt.xlabel('itérations')
plt.ylabel('erreur')
plt.plot(clf.loss_curve_)
plt.show()
```



On constate qu'après 1000 itérations la pente de la courbe n'est pas nulle, il est donc possible de diminuer encore l'erreur d'apprentissage en augmentant le nombre d'itérations.

Exploration : fonctions d'activation, nombre couches, nombre neurones par couche

Pour mieux comprendre l'utilisation des différentes fonctions d'activations proposées pour les neurones de la (des) couche(s) cachée(s), faisons des essais avec toutes les fonctions proposées, `identity`, `logistic`, `tanh`, `relu` (déjà employée ci-dessus), en conservant les autres paramètres (sauf `max_iter=1000`) aux valeurs par défaut. Remarquons que cela change le solveur utilisé, l'algorithme par défaut pour l'optimisation étant Adam (une variante adaptative de la descente de gradient).

```
# identity :
clflinear = MLPClassifier(activation='identity', max_iter=1000, random_state=100)
clflinear.fit(X_train1[:,2:4], y_train1)
train_score = clflinear.score(X_train1[:,2:4], y_train1)
print("Le score en train avec identity est {}".format(train_score))
test_score = clflinear.score(X_test1[:,2:4], y_test1)
print("Le score en test avec identity est {}".format(test_score))
#
# logistic :
clfsigmoid = MLPClassifier(activation='logistic', max_iter=1000, random_state=100)
clfsigmoid.fit(X_train1[:,2:4], y_train1)
train_score = clfsigmoid.score(X_train1[:,2:4], y_train1)
print("Le score en train avec logistic est {}".format(train_score))
test_score = clfsigmoid.score(X_test1[:,2:4], y_test1)
print("Le score en test avec logistic est {}".format(test_score))
#
# tangente hyperbolique :
clftanh = MLPClassifier(activation='tanh', max_iter=1000, random_state=100)
clftanh.fit(X_train1[:,2:4], y_train1)
train_score = clftanh.score(X_train1[:,2:4], y_train1)
print("Le score en train avec tanh est {}".format(train_score))
test_score = clftanh.score(X_test1[:,2:4], y_test1)
print("Le score en test avec tanh est {}".format(test_score))
```

Exercice 4 Que constatez-vous? Pourquoi les performances obtenues avec `identity` sont nettement inférieures? Il est utile pour cela de regarder quelle frontière de séparation est trouvée.

Exercice 5 Avec la fonction d'activation ReLU pour les neurones de la couche cachée, faites varier le nombre de neurones de cette couche entre 2 et 200. Que constatez-vous? Il peut être nécessaire d'augmenter la valeur de `max_iter` pour laisser l'apprentissage se poursuivre jusqu'à un minimum de l'erreur.

```
train_scores = []
test_scores = []
for hsize in [2, 10, 50, 100, 150, 200]:
    clf = MLPClassifier(hidden_layer_sizes=(hsize,), max_iter=3000, random_state=0)
    clf.fit(X_train1[:,2:4], y_train1)
    train_scores.append(clf.score(X_train1[:,2:4], y_train1))
    test_scores.append(clf.score(X_test1[:,2:4], y_test1))

plt.figure()
plt.xlabel('neurones_caches')
plt.ylabel('scores')
plt.plot([2, 10, 50, 100, 150, 200], train_scores, 'go')
plt.plot([2, 10, 50, 100, 150, 200], test_scores, 'r+')
plt.show()
```

Exercice 6 Avec la fonction d'activation ReLU pour les neurones des couches cachées et en conservant 10 neurones par couche cachée, faites varier le nombre de couches cachées entre 2 et 5. Que constatez-vous? Il peut être nécessaire d'augmenter la valeur de `max_iter` pour laisser l'apprentissage se poursuivre jusqu'à un minimum de l'erreur.